AI-Powered UAV for Precision Crop Monitoring and Yield Optimization

Shaik Naved Ahmed – 2303A54053
Mohammad Adil – 2303A54042
Patha Akash – 2303A54041
Ratnam Sanjay – 2303A54001

Submission Date: 19th November, 2025
Guide: Dr. Durgesh Nandan
Department: CS&AI, SR University, Warangal

Contents

Abstract 1

1 Introduction 2

2 Literature Review 3

3 Problem Statement 5

4 System Architecture 6

5 Methodology 8

6 Implementation 9

7 Results & Analysis 11

8 System Architecture Diagram 13

9 Discussion 14

10 Conclusion 15

11 Future Work 16

12 References 17

A Appendix A: Arduino Sketch (ESP32-CAM) 18

B Appendix B: Webhook Receiver (Flask) 22

Abstract

Artificial Intelligence (AI) is a key enabler for autonomy, perception and decision-making in Unmanned Aerial Vehicles (UAVs). This project surveys AI technologies relevant to UAVs and implements a low-cost ESP32-CAM prototype that demonstrates the perception pipeline: sensing → analysis → alert. The prototype captures images, runs a simulated anomaly detector, and sends JSON alerts via Wi-Fi to a webhook endpoint. Deliverables include a literature review, system architecture, methodology, implementation details, evaluation metrics, code listings, diagrams and a roadmap to scale the prototype into a UAV-deployable perception module. Empirical observations focus on latency, reliability under constrained power, and practical recommendations for moving from a simulated detector to on-device TinyML inference or an external edge accelerator.

Chapter 1

Introduction

Unmanned Aerial Vehicles (UAVs) are increasingly used across agriculture, infrastructure inspection, search and rescue, and surveillance. Modern UAVs require robust perception and decision-making capabilities to operate autonomously in dynamic environments. Artificial Intelligence (AI) provides the capabilities to turn raw sensor data into actionable insights, powering object detection, localization, mapping and autonomous navigation.

Motivation

High-cost sensors and heavy compute requirements limit prototyping for many academic teams. A low-cost perception prototype reduces entry barriers and accelerates iterative algorithm development.

Objectives

1. Survey AI methods employed in UAV perception and autonomy.
2. Design and implement a low-cost ESP32-CAM prototype that demonstrates the perception pipeline.
3. Measure and document prototype behavior, and provide a clear upgrade path to onboard inference and UAV integration.

Scope and Limitations

This work focuses on the perception and communication layer. Flight control, dynamics and in-air integration are outside this deliverable and planned as future extensions. The prototype uses simulated detection for robustness and reproducibility.

Chapter 2

Literature Review

This chapter summarizes prior work across the principal AI building blocks for UAVs.

Computer Vision

Object detection on UAVs commonly leverages single-stage detectors for real-time performance. The YOLO family (YOLOv3, YOLOv5, YOLOv8) and mobile-optimized networks (MobileNet, EfficientNet) are widely used depending on the compute budget.

Navigation and Planning

Techniques include classical planners (A*, RRT*), model-based controllers (MPC), and reinforcement learning methods (DQN, PPO) for adaptive navigation in complex scenarios.

Sensor Fusion and SLAM

Visual-Inertial Odometry (VIO) and SLAM approaches such as ORB-SLAM2 and VINS provide robust pose estimation by fusing camera and IMU measurements—critical for mapping and localization.

Edge Inference and TinyML

Deploying neural networks on microcontrollers requires quantization and model compression (TFLite, TensorFlow Lite Micro). Hardware accelerators (Coral TPU, NVIDIA Jetson) are alternatives for heavier models.

Gap Analysis

Many research systems assume powerful onboard compute and premium sensors. There is a gap for validated, reproducible, low-cost perception modules suitable for initial prototyping—motivating the ESP32-CAM approach used here.

Chapter 3

Problem Statement

Large-scale use of AI in UAVs depends on reliable perception modules. However, prototyping such modules is costly and complex. The problem addressed here is to create a reproducible, low-cost perception prototype that models the sensing and alerting behaviors necessary for UAV autonomy.

Why AI?

AI enables semantic interpretation, anomaly detection and automated alerting, which significantly reduce human workload and increase mission responsiveness.

Expected Improvement

A low-cost prototype reduces development time and clarifies integration steps needed to deploy perception modules on flight platforms.

Chapter 4

System Architecture

This chapter describes the prototype architecture and maps it to a UAV perception node.

Overview

The implemented system components are:

- **ESP32-CAM:** OV2640 camera with integrated microcontroller for image capture.
- **AI Detection Unit:** Simulated detection logic in firmware (future replacement with TinyML).
- **Wi-Fi Module:** Uses standard IEEE 802.11 for local connectivity.
- **Webhook Endpoint:** Cloud or local server that receives JSON alerts.
- **Monitoring Client:** Web/mobile client for log visualization.

Block Diagram

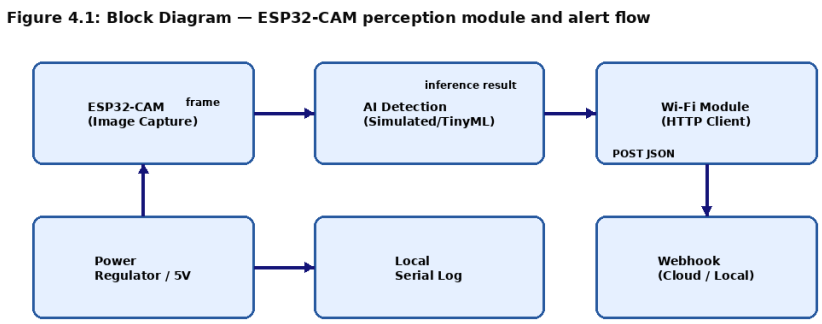

Figure 4.1: Figure 4.1: Block Diagram: ESP32-CAM perception module and alert flow.

Pipeline Mapping

The prototype models the perception node in a UAV, where outputs (detections/alerts) would integrate into a planner or flight controller node.

Chapter 5

Methodology

This section explains the design rationale and step-by-step approach used to build the prototype.

Design Goals

- **Modularity:** Keep detection, networking and logging modular to permit easy upgrades.
- **Reproducibility:** Use low-cost, widely available hardware and standard tools.
- **Extendability:** Make it trivial to swap simulated detection for a TinyML model.

Development Workflow

1. Assemble hardware and validate camera initialization.
2. Implement Wi-Fi client and webhook communication.
3. Implement simulated detection triggers and compact JSON payloads.
4. Create a Flask webhook receiver for validation and logging.
5. Execute bench and rover mobility tests while collecting logs and timing metrics.

Chapter 6

Implementation

Hardware Setup

Wire the ESP32-CAM to an FTDI USB-to-TTL adapter as follows:

- **5V** on ESP32-CAM ← VCC / 5V (FTDI or external 5V supply).
- **GND** ↔ GND (common ground).
- **U0R (RX)** on ESP32 ← FTDI TX.
- **U0T (TX)** on ESP32 → FTDI RX.
- **GPIO0** → GND only during flashing.

Power Recommendations Use a regulated 5V supply rated at 1–2 A to prevent voltage sag during camera initialization and Wi-Fi bursts.

Software Stack

- Arduino IDE with the ESP32 board support package.
- Libraries: `esp_camera.h`, `WiFi.h`, `HTTPClient.h`.
- Local webhook server implemented with Flask (Python).

Main Code Flow

1. Initialize camera with AI-Thinker pin mapping.
2. Connect to Wi-Fi and obtain IP.
3. Run simulated anomaly detection logic in loop.

4. When triggered, form a JSON payload and POST to webhook.
5. Log events to Serial Monitor for debugging and evaluation.

Workflow Diagram

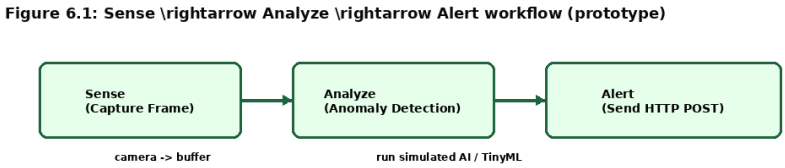

Figure 6.1: Figure 6.1: Sense → Analyze → Alert workflow (prototype).

Chapter 7

Results & Analysis

Test Setup

Testing consisted of:

- Bench tests with ESP32-CAM connected to laptop via FTDI.
- Flask webhook running locally on the laptop to receive alerts.
- Mobility tests using a low-cost rover carrying the ESP32 module.
- Serial Monitor at 115200 baud to capture logs.

Measured Metrics

Metric	Typical value	Notes
End-to-end alert latency	350–700 ms	Network dependent
Webhook HTTP response code	200	Flask returns acknowledgment
Detection trigger frequency	Simulated: 10 s	Parameter in firmware
Camera init reliability	95–100%	Requires stable 5V supply
Idle power draw	150–300 mA	Depends on Wi-Fi duty cycles

Table 7.1: Representative prototype performance metrics.

Representative Logs

```
[Serial] Camera initialized successfully.
[Serial] Connected. IP: 192.168.1.45
[Serial] Anomaly detected - sending alert.
[Webhook] 2025-11-18 12:23:45 Received: {"event":"ANOMALY","rssi":-52}
```

Discussion of Results

The results show the prototype can reliably produce and transmit alerts in near-real-time on local networks. The main limitations are the ESP32's constrained compute and memory for running modern CNNs and the variability of Wi-Fi links in non-lab environments.

Chapter 8

System Architecture Diagram

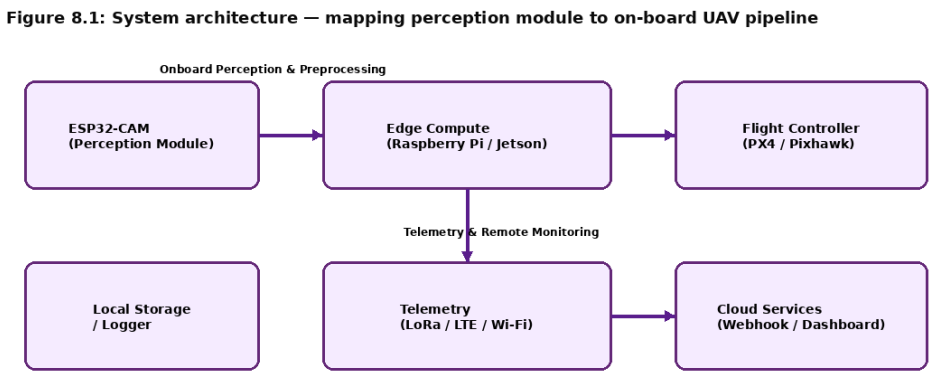

Figure 8.1: Figure 8.1: System architecture: mapping perception module to on-board UAV pipeline.

Chapter 9

Discussion

Key practical insights:

- Ensure a stable power supply; brownouts manifest as camera initialization failures.
- For real detection, adopt quantized TinyML models or an external edge board.
- For field deployments, replace Wi-Fi with LoRa/LTE for long-range telemetry.
- The modular firmware design makes it straightforward to replace simulated logic with a TFLite model.

Chapter 10

Conclusion

This project demonstrates that a low-cost ESP32-CAM prototype can emulate the perception and alerting layer of UAV systems. The work provides a practical foundation for researchers to develop, validate and iterate on perception algorithms before migrating them to flight-capable platforms.

Chapter 11

Future Work

1. Integrate a quantized TinyML detection model on the ESP32 or a slightly more capable edge device (Raspberry Pi + Coral TPU).
2. Add SLAM/VIO modules for localization and mapping.
3. Replace Wi-Fi with LoRa/LTE for long-range telemetry in field trials.
4. Integrate the perception module with a flight controller (PX4) for closed-loop autonomous behavior.

Chapter 12

References

1. J. Redmon and A. Farhadi, "YOLOv3: An Incremental Improvement," arXiv preprint, 2018.
2. Espressif Systems, "ESP32-CAM Datasheet," 2023.
3. PX4 Development Team, "PX4 Autopilot Documentation," 2024, https://px4.io.
4. J. Engel, T. Schops, and D. Cremers, "LSD-SLAM: Large-Scale Direct Monocular SLAM," ECCV, 2014.

Appendix A

Appendix A: Arduino Sketch (ESP32-CAM)

```
// ESP32-CAM Alert Prototype - Professional version
// NOTE: Change SSID, PASSWORD, and serverName before uploading.

#include "esp_camera.h"
#include "WiFi.h"
#include "HTTPClient.h"

// Configuration - change these values
const char* ssid = "YOUR_WIFI_NAME";
const char* password = "YOUR_WIFI_PASSWORD";
const char* serverName = "http://your-webhook.example.com/alert"
    ;

// AI-Thinker pinout (standard)
#define PWDN_GPIO_NUM     32
#define RESET_GPIO_NUM    -1
#define XCLK_GPIO_NUM      0
#define SIOD_GPIO_NUM     26
#define SIOC_GPIO_NUM     27
#define Y9_GPIO_NUM       35
#define Y8_GPIO_NUM       34
#define Y7_GPIO_NUM       39
#define Y6_GPIO_NUM       36
#define Y5_GPIO_NUM       21
#define Y4_GPIO_NUM       19
#define Y3_GPIO_NUM       18
#define Y2_GPIO_NUM        5
#define VSYNC_GPIO_NUM    25
#define HREF_GPIO_NUM     23
#define PCLK_GPIO_NUM     22

bool initCamera() {
    camera_config_t config;
    config.ledc_channel = LEDC_CHANNEL_0;
```

```
    config.ledc_timer   = LEDC_TIMER_0;
    config.pin_d0       = Y2_GPIO_NUM;
    config.pin_d1       = Y3_GPIO_NUM;
    config.pin_d2       = Y4_GPIO_NUM;
    config.pin_d3       = Y5_GPIO_NUM;
    config.pin_d4       = Y6_GPIO_NUM;
    config.pin_d5       = Y7_GPIO_NUM;
    config.pin_d6       = Y8_GPIO_NUM;
    config.pin_d7       = Y9_GPIO_NUM;
    config.pin_xclk     = XCLK_GPIO_NUM;
    config.pin_pclk     = PCLK_GPIO_NUM;
    config.pin_vsync    = VSYNC_GPIO_NUM;
    config.pin_href     = HREF_GPIO_NUM;
    config.pin_sscb_sda = SIOD_GPIO_NUM;
    config.pin_sscb_scl = SIOC_GPIO_NUM;
    config.pin_pwdn     = PWDN_GPIO_NUM;
    config.pin_reset    = RESET_GPIO_NUM;
    config.xclk_freq_hz = 20000000;
    config.pixel_format = PIXFORMAT_JPEG;

    if (psramFound()) {
        config.frame_size = FRAMESIZE_QVGA;
        config.jpeg_quality = 12;
        config.fb_count = 2;
    } else {
        config.frame_size = FRAMESIZE_QQVGA;
        config.jpeg_quality = 15;
        config.fb_count = 1;
    }

    esp_err_t err = esp_camera_init(&config);
    if (err != ESP_OK) {
        Serial.printf("Camera init failed: 0x%x\n", err);
        return false;
    }
    Serial.println("Camera initialized successfully.");
    return true;
}

bool anomalyDetected() {
    return (millis() % 10000UL) < 500UL;
}

void sendAlert(const String &message) {
    if (WiFi.status() != WL_CONNECTED) {
        Serial.println("WiFi not connected - alert skipped.");
        return;
    }
    HTTPClient http;
    http.begin(serverName);
    http.addHeader("Content-Type", "application/json");
```

```
    String payload = "{\"event\":\"ANOMALY\",\"message\":\"" + message + "\",\"rssi\":" + String(WiFi.RSSI()) + "}";
    int statusCode = http.POST(payload);
    Serial.printf("POST status: %d\n", statusCode);
    http.end();
}

void connectWiFi() {
    Serial.printf("Connecting to WiFi '%s' ...\n", ssid);
    WiFi.mode(WIFI_STA);
    WiFi.begin(ssid, password);
    unsigned long start = millis();
    while (WiFi.status() != WL_CONNECTED && (millis() - start) < 20000UL) {
        Serial.print(".");
        delay(250);
    }
    Serial.println();
    if (WiFi.status() == WL_CONNECTED) {
        Serial.printf("Connected. IP: %s\n", WiFi.localIP().toString().c_str());
    } else {
        Serial.println("WiFi connection failed.");
    }
}

void setup() {
    Serial.begin(115200);
    delay(200);
    if (!initCamera()) {
        Serial.println("Camera init failed - continuing without camera.");
    }
    connectWiFi();
    Serial.println("Prototype ready.");
}

void loop() {
    static unsigned long lastWifiCheck = 0;
    if (WiFi.status() != WL_CONNECTED && millis() - lastWifiCheck > 15000UL) {
        connectWiFi();
        lastWifiCheck = millis();
    }

    if (anomalyDetected()) {
        Serial.println("Anomaly detected - sending alert.");
        sendAlert("Simulated anomaly detected");
        delay(15000);
    } else {
        if ((millis() % 30000UL) < 500UL) {
```

```
            Serial.println("Monitoring...");
        }
    }
    delay(100);
}
```

Appendix B

Appendix B: Webhook Receiver (Flask)

```python
# webhook_receiver.py - simple Flask webhook receiver
from flask import Flask, request, jsonify
import datetime

app = Flask(__name__)

@app.route('/alert', methods=['POST','GET'])
def alert():
    data = request.get_json() or request.args.to_dict()
    ts = datetime.datetime.now().strftime('%Y-%m-%d %H:%M:%S')
    print(f"[{ts}] ALERT RECEIVED: {data}")
    return jsonify({"status":"ok","received":data,"timestamp":ts
        })

if __name__ == '__main__':
    print("Webhook listening on http://0.0.0.0:5000/alert")
    app.run(host='0.0.0.0', port=5000)
```